Read-Write Lock

son: 5 · Concurrency · Category: Concurrency · Level: Intermediate · Build with Wa

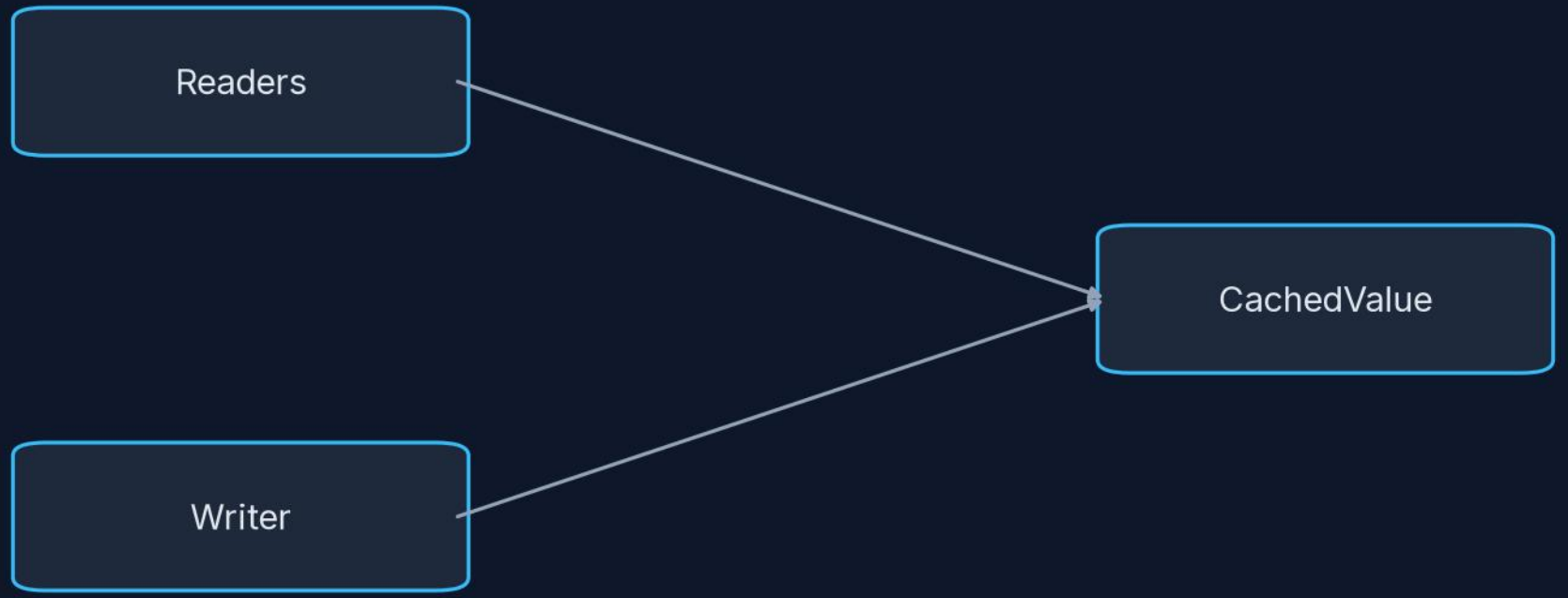
Today's Hook

- Today: Read-Write Lock as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Read-Write Lock structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Read-Write Lock
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/concurrency/readwritelock/CachedValue.java
package com.buildwithwaqar.patterns.concurrency.readwritelock;

import java.util.concurrent.locks.ReentrantReadWriteLock;
import java.util.function.Supplier;

public final class CachedValue<T> {
    private final ReentrantReadWriteLock lock = new ReentrantReadWriteLock();
    private T value;

    public T get(Supplier<T> loader) {
        lock.readLock().lock();
        try {
            if (value != null) return value;
        } finally {
            lock.readLock().unlock();
        }
        lock.writeLock().lock();
        try {
            if (value == null) value = loader.get();
            return value;
        } finally {
            lock.writeLock().unlock();
        }
```

Implementation (2)

```
    }

    public void clear() {
        lock.writeLock().lock();
        try { value = null; } finally { lock.writeLock().unlock(); }
    }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.functional.writer;
import com.buildwithwaqar.patterns.functional.monoid.Monoids;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Writer walkthrough ===");
    var m = Monoids.STRING;
    Writer<String, Integer> w = Writer.<String, Integer>pure(m, 1)
      .flatMap(m, n -> new Writer<>("inc;", n + 1))
      .flatMap(m, n -> new Writer<>("dbl;", n * 2));
    System.out.println("value=" + w.value() + " log=" + w.log());
  }
}
```

Walkthrough Output

```
=== Read-Write Lock walkthrough ===
manifest-v1
manifest-v1
loads=1
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Read-Write Lock
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Read-Write Lock → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.